

FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P1

Relatório bimestral de atividades

Avaliação individual · 2026.2

Pedro Barbosa Ferreira

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 07/10/2026

Sumário

1	Introdução	2
2	Metodologia	2
3	Quadro de entregas	3
4	Atividades realizadas	3
4.1	Atividade 1 — Agenda telefônica em Flask	3
4.2	Atividade 2 — Catálogo de filmes — Tom Hanks	5
4.3	Atividade 3 — Desacoplando o login — microsserviço de autenticação	6
4.4	Atividade 4 — Controle de acesso por papel — RBAC	9
4.5	Atividade 5 — Logs e auditoria	12
4.6	Atividade 6 — Upload e perfil de usuário	14
5	Considerações finais	16
6	Declaração de autoria	16

1 Introdução

Este relatório documenta os trabalhos realizados na disciplina Computação em Nuvem, cujo o objetivo não é so de pratica mas sim introduzindo conceitos e partindo de um ambiente real, cada aluno tem seu banco de dados relacional, com controle de acesso isolado dos demais alunos, da mesma maneira que um provedor de nuvem gerencia contas de cliente diferentes dentro do mesmo serviço gerenciado. A disciplina discute os fundamentos que sustenta, qualquer arquitetura em nuvem

O projeto que foi o fio condutor do bimestre consiste em uma aplicação baseada na filmologia de Tom Hanks, consumindo em tempo real a API publica do TMDb. A aplicação consiste em características onde os usuários se cadastram, navegam por um catalogo de filmes, favoritam títulos, deixam seus comentários, tem um perfil próprio onde o pode-se inserir foto de perfil, bio, entre outras funções.

Para garantir a governança e a segurança da informação, a arquitetura conta com controle de acesso baseado em papéis (Role-Based Access Control — RBAC), um microsserviço dedicado de auditoria gravando eventos em tempo real via Redis Streams, e um repositório de objetos (Object Storage) gerenciado pelo MinIO para armazenamento privado de imagens. Este relatório consolida e comprova a execução de todas as etapas planejadas e entregues no período.

2 Metodologia

O desenvolvimento seguiu uma abordagem evolutiva e incremental. Cada atividade do bimestre foi feita com base no código da atividade anterior, formando um único projeto em constante evolução. A containerização com Docker e Docker Compose foi usada desde o início, assim que a arquitetura exigiu vários serviços. Isso fez com que o ambiente de desenvolvimento já tivesse o isolamento que o sistema teria em produção.

A divisão em microsserviços seguiu um critério claro: sempre que uma nova exigência tinha uma responsabilidade diferente das que já existiam, como identidade, auditoria, armazenamento de arquivos, ela foi colocada em um novo serviço separado. Serviços que não precisavam de acesso externo ficaram sem porta publicada, restritos à rede interna do Docker. Só um ponto de entrada público era acessível, responsável por todas as decisões de autorização. Cada funcionalidade só foi considerada pronta depois de testada em ambos os sentidos com acesso autorizado e com acesso negado, o segundo simulado com curl, Postman ou diretamente no formulário.

O trabalho todo foi feito em um único repositório público no GitHub. Começou com um ambiente GitHub Codespaces com Docker Compose. Para o banco MySQL externo, usou-se DBeaver. Para o deploy, usou-se Portainer. Para validar os endpoints, usou-se navegador, curl e Postman. Houve apoio nas documentações oficiais de cada tecnologia usada, as tecnologias foram: Flask, MySQL Connector, Redis, MinIO, flaskgger, Unicorn, Brevo.

As datas e horários de entrega foram obtidos do histórico de commits, conferidos no GitHub. As evidências foram coletadas com capturas de tela do sistema rodando, logs no Portainer, consultas no DBeaver e chamadas no Swagger UI. A data e a hora de cada entrega foram extraídas do histórico de commits (git log) e conferidas na página do commit no GitHub. Os prints mostram o README com a menção ao professor, o histórico de commits e o sistema em execução.

3 Quadro de entregas

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
1	Agenda telefônica em Flask	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.	07/08/2026	04/10/2026 19:37	entregue com atraso
2	Catálogo de filmes — Tom Hanks	Consumo da API TMDb, persistência em MariaDB e segregação por usuário.	20/08/2026	20/08/2026 22:40	entregue
3	Desacoplando o login — microserviço de autenticação	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.	28/08/2026	25/08/2026 16:28	entregue
4	Controle de acesso por papel — RBAC	O campo role passa a decidir permissões reais no backend (403 para usuário comum).	04/09/2026	02/09/2026 09:15	entregue
5	Logs e auditoria	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.	25/09/2026	17/09/2026 22:20	entregue
6	Upload e perfil de usuário	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.	02/10/2026	30/09/2026 09:57	entregue

4 Atividades realizadas

4.1 Atividade 1 — Agenda telefônica em Flask

Descrição	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.
Data planejada	07/08/2026
Data realizada	04/10/2026 19:37
Situação	entregue com atraso
Evidência	Realizada em sala — print do sistema rodando localmente (e do repositório, se foi versionado)
Link	https://github.com/PedroBarbosa42/agenda-telefonica

O que foi feito. Foi feita uma agenda telefônica simples como aplicação monolítica, usando Flask e templates Jinja2 em um único projeto, sem separação entre front-end e back-end. O sistema permite cadastrar contatos (nome, telefone e e-mail), listar todos os contatos e consultar os detalhes de cada um. A persistência foi feita em um arquivo contatos.json, lido e gravado a cada requisição.

← → ↺ 🏠 🌐 ideal-invention-g46vrvqvrqvgf95v

[Contatos](#) | [Novo contato](#)

Novo contato

Nome:

Telefone:

E-mail:

← → ↺ 🏠 🌐 ideal-invention-g46vrvqvrqvgf95v

[Contatos](#) | [Novo contato](#)

Contatos

- [Pedro](#) — 14998625884

🌱 agenda-telefonica Public

📁 main 1 Branch 0 Tags

🔍 Go to file T Add file >> Code

🌱 PedroBarbosa42 Feito fora de prazo para entrega p1 3811783 · yesterday 3 Commits

📁 templates	Feito	5 days ago
📄 README.md	Feito fora de prazo para entrega p1	yesterday
📄 app.py	Feito	5 days ago
📄 contatos.json	Feito	5 days ago
📄 requirements.txt	Feito	5 days ago

📖 README

Agenda-telefonica

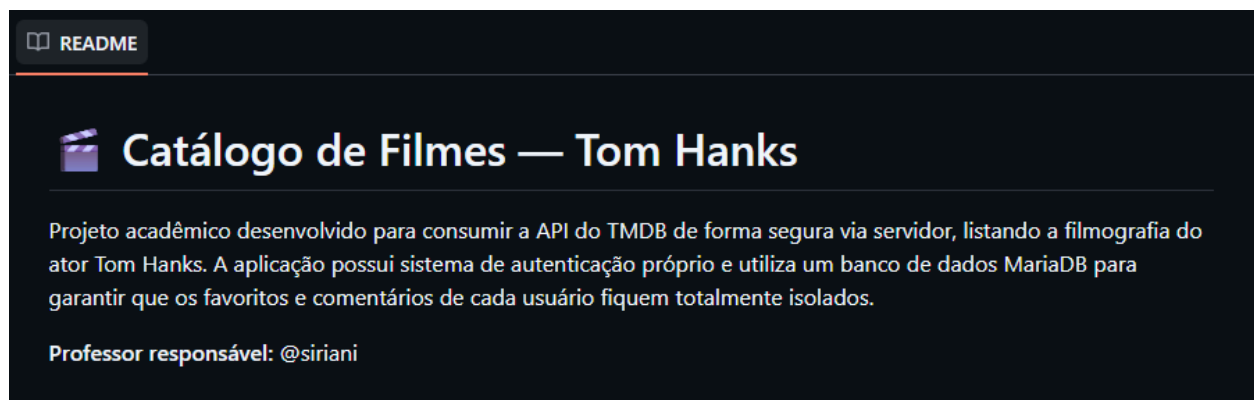
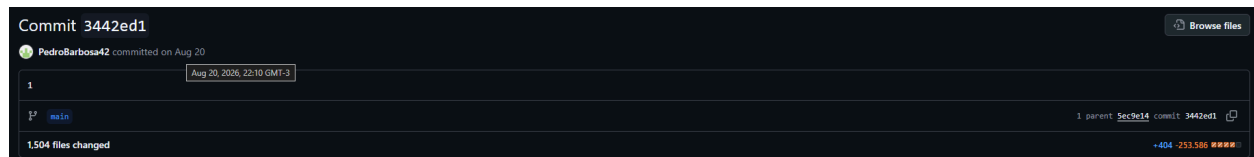
Agenda telefônica simples como aplicação monolítica, usando Flask e templates Jinja2 em um único projeto, sem separação entre front-end e back-end. O sistema permite cadastrar contatos (nome, telefone e e-mail), listar todos os contatos e consultar os detalhes de cada um. A persistência foi feita em um arquivo contatos.json, lido e gravado a cada requisição. @siriani

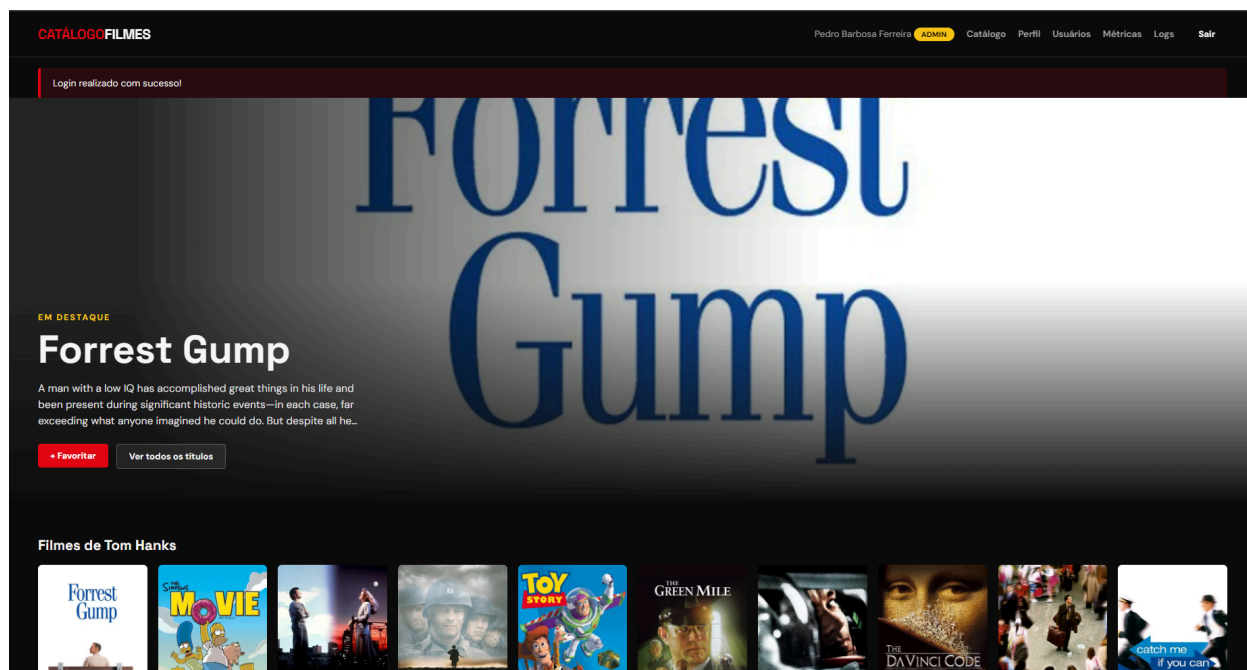
Dificuldades e como foram resolvidas. A principal dificuldade foi fazer o projeto do zero fora da sala de aula, o que exigiu montar sozinho o repositório, o ambiente virtual e a estrutura de pastas. Foi buscado a ajuda de amigos e da Inteligência Artificial.

4.2 Atividade 2 — Catálogo de filmes — Tom Hanks

Descrição	Consumo da API TMDB, persistência em MariaDB e segregação por usuário.
Data planejada	20/08/2026
Data realizada	20/08/2026 22:40
Situação	entregue
Evidência	GitHub — commit + README + print do catálogo
Link	https://github.com/PedroBarbosa42/CatalogoFilmes

O que foi feito. Nesta atividade, desenvolveu-se a primeira versão funcional da aplicação web do Catálogo de Filmes com arquitetura cliente-servidor em Flask. Foi implementado o consumo de API pública do TMDB diretamente no backend em Python/Flask, autenticação e a segregação de dados, onde todas as consultas SQL filtram o acesso obrigatoriamente pelo `usuario_id` da sessão ativa, modelação e persistência no banco de dados, containerização e segurança de credenciais.





Dificuldades e como foram resolvidas. O principal desafio técnico nesta etapa foi garantir a segurança das credenciais da API do TMDb e do banco de dados num repositório público do GitHub. Centralizou-se todo o consumo de APIs no servidor, ocultaram-se as chaves via .env.

4.3 Atividade 3 — Desacoplando o login — microsserviço de autenticação

Descrição	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.
Data planejada	28/08/2026
Data realizada	25/08/2026 16:28
Situação	entregue
Evidência	GitHub — commit + docker-compose.yml + print do login funcionando
Link	https://github.com/PedroBarbosa42/CatalogoFilmes

O que foi feito. Extraíu-se toda a lógica de autenticação e gestão de usuários do monólito para um microsserviço independente em container Docker separado (auth_api). O serviço foi isolado na rede interna do Docker Compose sem portas expostas ao host, mantendo o catalogo_web como a única porta de entrada pública do sistema. Implementou-se a gestão de papéis (role) para diferenciar permissões entre usuario e admin, (Ainda não implementado no visual, somente a logica). Também foi criado o fluxo de recuperação de senha com a tabela reset_tokens, gerando tokens hexadecimais de 32 bytes válidos por 30 minutos e disparados por e-mail SMTP via Brevo, (Primeiramente foi feita um simulado pelo Mailtrap, posteriormente foi utilizado o Brevo para a recuperação por e-mail real).

Commit 6b18aa1

 **PedroBarbosa42** committed on Aug 25

24/08 (2)

Aug 25, 2026, 00:08 GMT-3

 **main**

14 files changed



 README.md

 auth_service

 app.py

 catalogo_service

 app.py

 templates

 forgot_password.html

 login.html

 reset_password.html

 **PedroBarbosa42** opened on [Aug 25](#)Owner

Aug 25, 2026, 16:28 GMT-3

Catálogo de Filmes - Microsserviços e Autenticação

Atualização da arquitetura para a Atividade 3, implementando um serviço isolado de autenticação e comunicação via rede interna do Docker.

Professor: [@siriani](#)

 O que mudou na Arquitetura?

Serviço de Autenticação Isolado: Todo o fluxo de login, cadastro, papéis de usuário (role) e recuperação de senha foi extraído do catálogo e movido para um novo container dedicado (auth_api).

Rede Interna Segura: O serviço de autenticação não possui portas publicadas para o host. O catálogo é o único ponto de entrada público. Toda a comunicação entre o catálogo e a autenticação ocorre invisível para a internet, utilizando o DNS interno do Docker.

Recuperação de Senha com SMTP Real: Implementação da geração de tokens seguros (32 bytes) com expiração de 30 minutos, armazenados no banco de dados. O envio de e-mails de recuperação é feito de forma real via porta 2525 utilizando o ambiente de desenvolvimento Mailtrap.

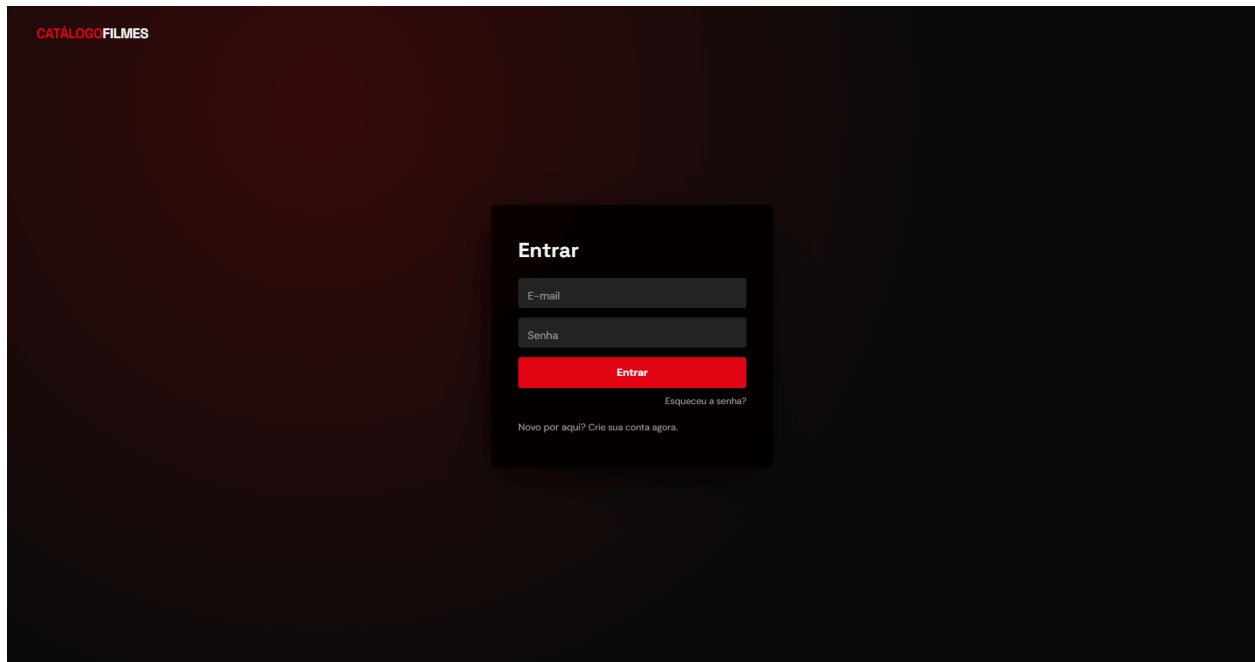
 Configuração do Docker (docker-compose.yml)

Abaixo, a demonstração de como os serviços estão isolados na mesma rede, garantindo que o auth_api não exponha portas externas:

```
services:
  catalogo_web:
    build: ./catalogo_service
    ports:
      - "8225:5000" # ÚNICO PONTO PÚBLICO
    environment:
      - FLASK_SECRET_KEY=${FLASK_SECRET_KEY}
    depends_on:
      - db
      - auth_api
    networks:
      - rede_catalogo

  auth_api:
    build: ./auth_service
    # SEM PORTAS EXPOSTAS (sem diretiva 'ports')
    environment:
      - DB_HOST=${DB_HOST}
      - DB_USER=${DB_USER}
      - DB_PASSWORD=${DB_PASSWORD}
      - DB_NAME=${DB_NAME}
      - MAIL_SERVER=${MAIL_SERVER}
      - MAIL_PORT=${MAIL_PORT}
      - MAIL_USERNAME=${MAIL_USERNAME}
      - MAIL_PASSWORD=${MAIL_PASSWORD}
    depends_on:
      - db
    networks:
      - rede_catalogo

networks:
  rede_catalogo:
    driver: bridge
```



Dificuldades e como foram resolvidas. Configuração do Brevo, a conta foi bloqueada sem motivos claros, mantive o Mailtrap por um tempo até o suporte responder mas sem sucesso, foi criado uma conta somente para configuração do Brevo.

4.4 Atividade 4 — Controle de acesso por papel — RBAC

Descrição	O campo role passa a decidir permissões reais no backend (403 para usuário comum).
Data planejada	04/09/2026
Data realizada	02/09/2026 09:15
Situação	entregue
Evidência	GitHub — commit + print do 403 e da ação de admin
Link	https://github.com/PedroBarbosa42/CatalogoFilmes

O que foi feito. Foi desenvolvido funcionalidades exclusivas de administração no backend, incluindo a moderação global de comentários, o painel de gestão de papéis (/admin/usuarios), a eliminação de contas e a consulta de métricas do sistema.

Estabeleceu-se a validação estrita de permissões no lado do servidor através do decorator `admin_required`, garantindo o bloqueio com código HTTP 403 Forbidden a qualquer tentativa de acesso não autorizado, inclusive via chamadas diretas com Postman ou cURL.

Commit a642942

 PedroBarbosa42 committed on Sep 1

01/09

 main

2 files changed

- auth_service
 - app.py
- catalogo_service
 - app.py

Commit f440a83

 PedroBarbosa42 committed on Sep 1

01/09

 main

4 files changed

- catalogo_service
- static
 - style.css
- templates
 - admin_metricas.html
 - admin_usuarios.html
 - index.html

Commit 0c2c4e1

 PedroBarbosa42 committed on Sep 1

01/09

 main

1 file changed

- README.md

The image displays a web application interface and two test results. The top section shows the application's main page, 'Filmes de Tom Hanks', with a dark blue background and white text. It includes a navigation bar with 'Gestão de papéis', 'Métricas', and 'Sair' buttons. A message 'Login realizado com sucesso!' is visible. Below this, two test results are shown for the endpoints `/admin/usuarios` and `/admin/metricas`.

Teste `/admin/usuarios`:

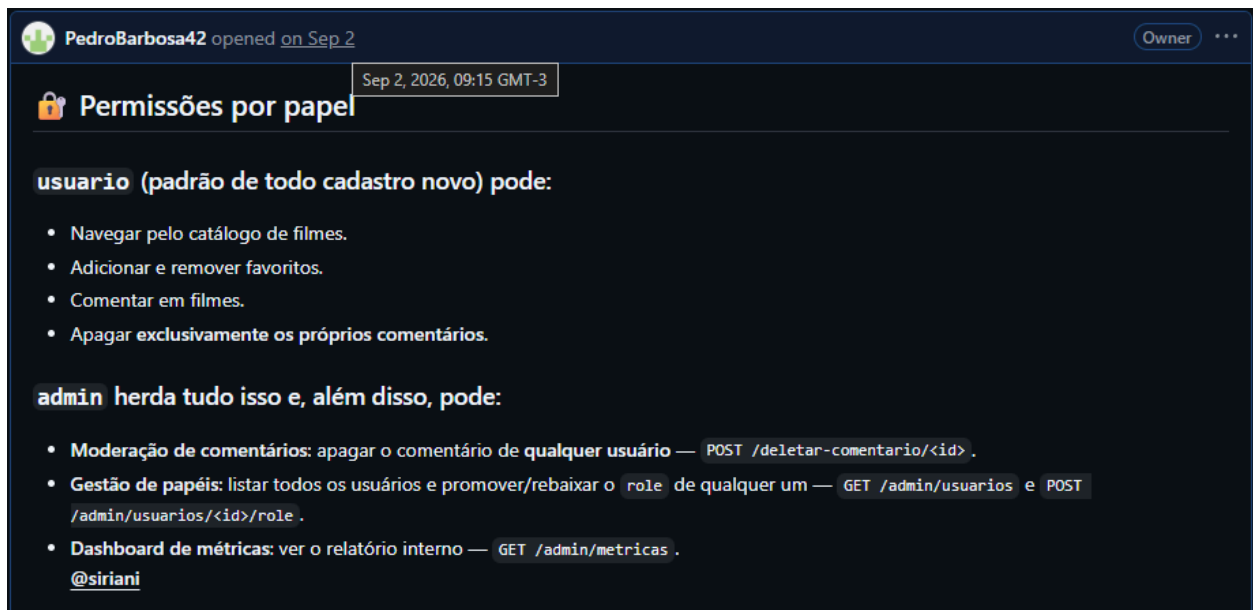
- Usuário comum → 403**: The test result shows a 403 status code, indicating that a common user is not allowed to access this endpoint.
- Admin → sucesso**: The test result shows a successful response (200) for an admin user. The response body is a JSON array of user data:

ID	nome	email	status	função
1	Pedro Barbosa Ferreira	pedro@isw055.com	ativo	admin
2	Maria	maria@isw055.com	inativo	usuario
3	João	joao@isw055.com	inativo	usuario
4	Carlos	carlos@isw055.com	inativo	usuario

O mesmo teste, repetido para `/admin/metricas`:

- Usuário comum → 403**: The test result shows a 403 status code, indicating that a common user is not allowed to access this endpoint.
- Admin → sucesso**: The test result shows a successful response (200) for an admin user. The response body is a JSON object representing a dashboard with metrics:

Métrica	Valor
Usuários ativos	4
Usuários inativos	8
Filmes disponíveis	4
Comentários publicados	0



Dificuldades e como foram resolvidas. Evitar a falha de segurança comum de restringir acessos apenas ocultando botões na interface gráfica do frontend. Utilizado o papel do utilizador no backend em todas as rotas sensíveis utilizando os dados guardados na sessão assinada pelo Flask (SECRET_KEY), recusando pedidos não autorizados diretamente na API.

4.5 Atividade 5 — Logs e auditoria

Descrição	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.
Data planejada	25/09/2026
Data realizada	17/09/2026 22:20
Situação	entregue
Evidência	GitHub — commit + print da consulta de logs pelo admin
Link	https://github.com/PedroBarbosa42/CatalogoFilmes

O que foi feito. Desenvolveu-se um novo microserviço dedicado de auditoria (log_service), isolado num container próprio na rede interna do Docker Compose sem portas expostas para a internet. Para a persistência dos eventos, configurou-se um container Redis 7 Alpine utilizando Redis Streams, gravando os registros ordenados temporalmente com o comando XADD e lendo o histórico recente via XREVRANGE. O catalogo_web foi instrumentado para disparar requisições contendo usuario_id, acao, detalhe, ip e timestamp, cobrindo ações como login, logout, favoritar, comentar, apagar_comentario, edições de perfil e bloqueios com acesso_negado (HTTP 403). Por fim, criou-se a tela de consulta /admin/logs no painel administrativo, protegida pelo decorator admin_required, para exibição dos registros em ordem cronológica.

Commit d2b8a1e

 **PedroBarbosa42** committed 3 weeks ago

Sep 15, 2026, 21:23 GMT-3

Logs e auditoria

 **main**

9 files changed

Q Filter files...



▼  **catalogo_service**

 **app.py**

▼  **templates**

 **admin_logs.html**

 **admin_metricas.html**

 **admin_usuarios.html**

 **index.html**

 **docker-compose.yml**

▼  **log_service**

 **Dockerfile**

 **app.py**

 **requirements.txt**


PedroBarbosa42 opened 2w ago
Owner

Sep 17, 2026, 22:20 GMT-3

Título

Atividade 5 — Exclusão de usuário (admin) + Logs de auditoria com Redis
@siriani

Descrição

Continuação direta da atividade de controle de acesso (Atividade 4). Duas entregas nesta etapa:

1. Exclusão de usuário (admin)

- Nova rota interna no `auth_service`: `DELETE /usuarios/<id>` — apaga favoritos, comentários e tokens de reset do usuário antes de apagar o usuário em si (evita violar as chaves estrangeiras).
- Nova rota pública no `catalogo_service`: `POST /admin/usuarios/<id>/deletar`, protegida por `@admin_required`.
- Um admin **não pode apagar a própria conta** enquanto estiver logado (bloqueio explícito, evita se auto-excluir e ficar sem acesso).
- Botão "Apagar" adicionado na tela `/admin/usuarios`, com confirmação (`confirm()`) antes de enviar, escondido na própria linha do admin logado.

2. Logs e auditoria

- Novo microserviço `log_service`: container próprio, mesma rede interna do Docker, **sem porta publicada pro host** — mesmo princípio do `auth_service`.
- Novo serviço `redis` no `docker-compose.yml`, também sem porta publicada, com volume persistente.
- Eventos registrados via Redis Streams (XADD): `login`, `logout`, `favoritar`, `desfavoritar`, `comentar`, `apagar_comentario` (com detalhe indicando se foi próprio ou moderação), e `acesso_negado` (toda tentativa de ação de admin barrada por um usuário comum).
- Cada evento grava `usuario_id`, `acao`, `detalhe`, `ip` e `timestamp`.
- Nova rota `GET /admin/logs` no `catalogo_service`, protegida pelo mesmo `@admin_required`, consultando os últimos N eventos via `XREVRANGE` (mais recente primeiro). Timestamp exibido já convertido pro horário de Brasília (UTC-3).

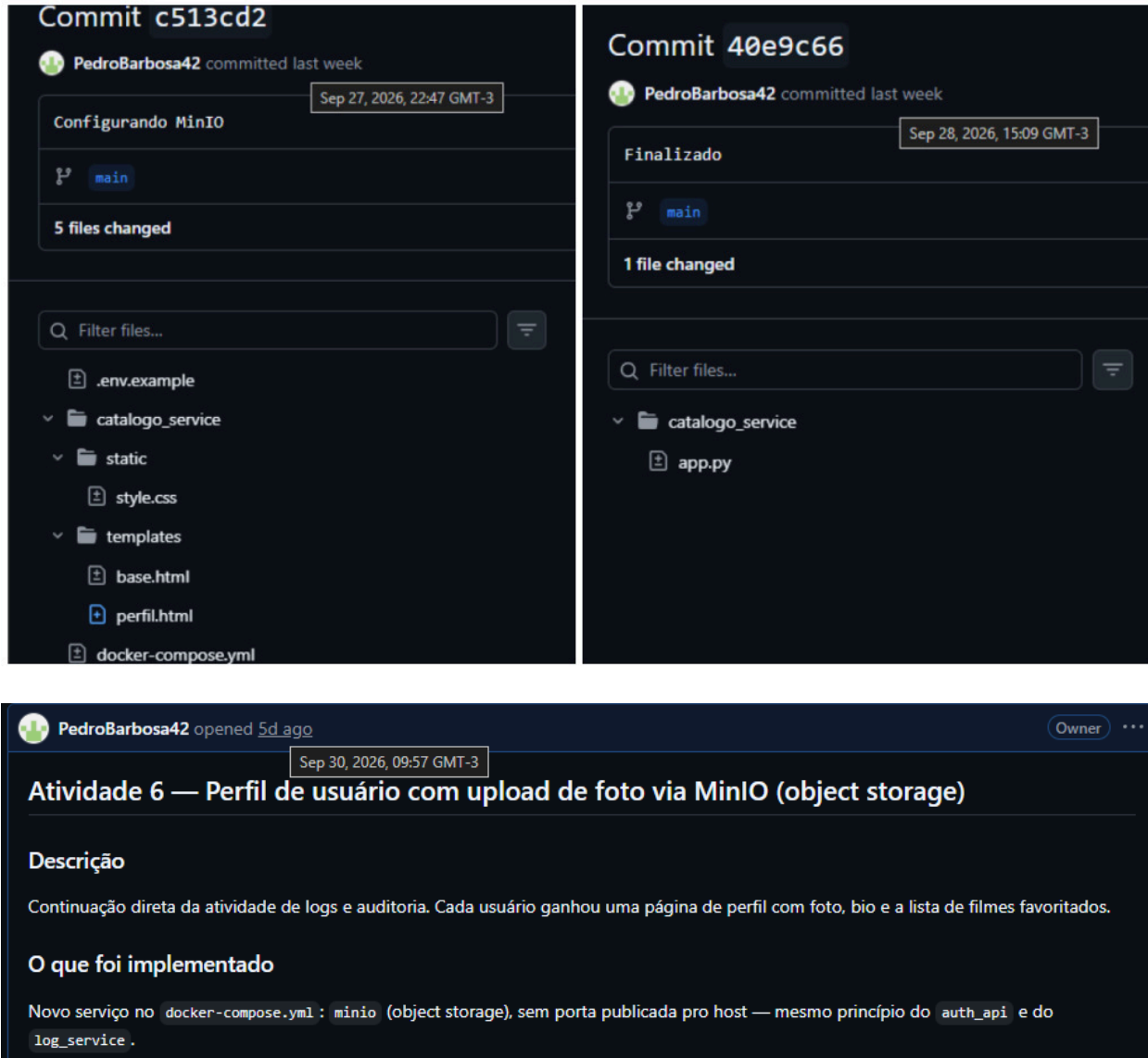
15/09/2026 20:58:29	2	LOGOUT		172.18.0.1
15/09/2026 20:58:06	2	ACESSO_NEGADO	tentou acessar /admin/usuarios	172.18.0.1
15/09/2026 20:57:41	2	COMENTAR	Não gostei do filme!	172.18.0.1
15/09/2026 20:57:31	2	FAVORITAR	The Simpsons Movie	172.18.0.1
15/09/2026 20:57:08	2	LOGIN		172.18.0.1

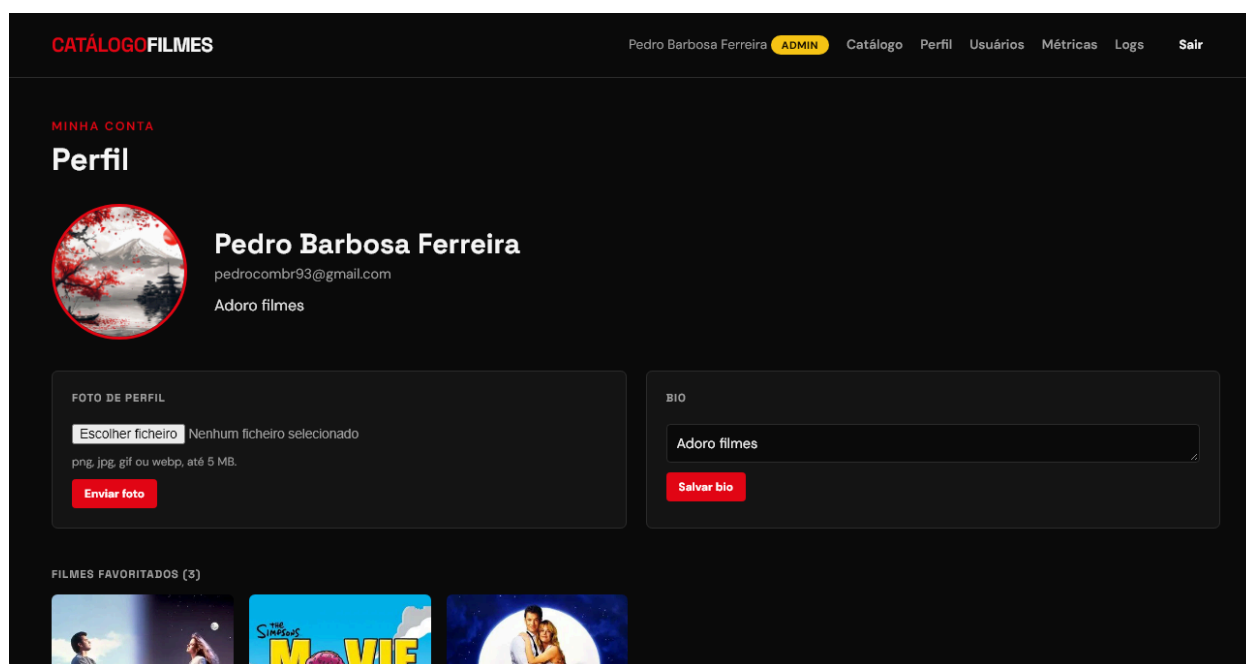
Dificuldades e como foram resolvidas. Uma das dificuldades enfrentadas foi a exibição dos timestamps gravados em UTC no Redis de forma adequada ao fuso horário local (Horário de Brasília / UTC-3). A solução consistiu em formatar e converter as datas diretamente no backend do `catalogo_web` antes da renderização na interface.

4.6 Atividade 6 — Upload e perfil de usuário

Descrição	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.
Data planejada	02/10/2026
Data realizada	30/09/2026 09:57
Situação	entregue
Evidência	GitHub — commit + print do perfil com foto
Link	https://github.com/PedroBarbosa42/CatalogoFilmes

O que foi feito. Foi desenvolvido a página de perfil de usuário reunindo nome, bio, foto e a lista de filmes favoritos. Implementou-se o envio da imagem de perfil para um object storage MinIO dedicado, validando o formato (apenas imagens) e o tamanho limite do arquivo no servidor antes de concluir o envio. O banco de dados MariaDB passou a armazenar unicamente a chave de referência do arquivo, evitando o inchaço do banco relacional com dados binários.





Dificuldades e como foram resolvidas. A principal dificuldade técnica nesta etapa esteve associada ao deploy do serviço MinIO no ambiente Docker. Inicialmente, o Docker Compose falhou ao tentar efetuar o pull da imagem pública minio/minio devido a restrições do registro (pull access denied). Após a alteração da imagem, surgiu um conflito de alocação de portas no servidor, uma vez que a porta 9000 já se encontrava em uso por outro serviço (port is already allocated). O problema foi resolvido atualizando o docker-compose.yml para utilizar uma imagem alternativa e funcional (pgsty/minio:RELEASE.2026-08-04T00-00-00Z) e removendo a publicação da porta 9000 para fora do host, mantendo o tráfego do MinIO restrito à rede interna do Docker e alterando o endereço da consola de gestão para a porta :9001.

5 Considerações finais

Esta disciplina destacou-se por ter sido a única que nos colocou diante de um projeto prático e real do início ao fim. Por conta dessa exigência prática, acabamos ficando um pouco perdidos no começo, mas ao longo do processo foi possível compreender métodos essenciais de arquitetura de software e identificar falhas comuns que costumamos cometer no início do aprendizado.

Olhando para trás, o que eu faria diferente seria ter tentado buscar uma postura mais autônoma desde os primeiros desafios, acelerando a resolução dos problemas e a compreensão da infraestrutura. Pretendo levar esse aprendizado adiante, meu objetivo é continuar aprimorando esta aplicação, aplicando novas melhorias e estudando a fundo a estrutura que foi construída até aqui.

6 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados. Nas atividades realizadas em grupo, o conteúdo aqui descrito refere-se à minha participação.

Pedro Barbosa Ferreira

Pompeia, 07/10/2026